**Fundación Universitaria Konrad Lorenz**

Ingeniería de Sistemas

Nuevas Tecnologías de Desarrollo

# Entrega 2: Integración de Datos No Estructurados (NoSQL)

Sistema de Gestión de Inventario para Gastrobar

Justificación de la estructura de datos en MongoDB

**Juan Esteban Pico Briceño**

Bogotá D.C., noviembre de 2026

## 1. Contexto del proyecto

El proyecto es una API en Flask para manejar el inventario de un gastrobar: qué productos hay (cervezas, licores, bebidas, insumos de cocina y desechables), quién los provee, cuánto hay en existencia y cómo se mueve ese inventario día a día.

En la primera parte la aplicación trabajaba solo con una base de datos relacional en MySQL, con tres tablas: **categorias**, **proveedores** y **productos**. Eso funciona bien para el catálogo, pero quedaban por fuera dos cosas que en el negocio pasan todo el tiempo: el registro de cada entrada y salida de mercancía, y las características propias de cada tipo de producto, que no son las mismas para una cerveza que para un paquete de chorizos.

En esta entrega se agregó MongoDB para manejar esa información y la aplicación pasó a trabajar con **persistencia políglota**: cada tipo de dato se guarda en la base que mejor se le ajusta y la API consulta las dos según lo que necesite.

## 2. Identificación de los datos no estructurados

Para decidir qué se movía a NoSQL revisé cada dato del problema con cuatro preguntas: ¿tiene siempre los mismos campos?, ¿necesita relaciones e integridad fuerte?, ¿cuántos registros se generan? y ¿cómo se consulta normalmente?

| Dato | ¿Estructura fija? | Volumen | Decisión |
|---|---|---|---|
| Categorías, proveedores | Sí | Bajo, casi no cambia | MySQL |
| Producto (nombre, precios, stock, mínimos) | Sí | Bajo | MySQL |
| Atributos de cada producto (grados de alcohol, ml, alérgenos, vida útil, material...) | No: cambian según el tipo de producto | Bajo | MongoDB – fichas_producto |
| Movimientos de inventario (entradas, salidas, mermas, ajustes) | Parcial: la base es igual pero el detalle cambia según el tipo | Alto: cada venta genera uno | MongoDB – movimientos_inventario |
| Registro de actividad (quién hizo qué) | No: cada acción guarda datos distintos | Alto | MongoDB – logs_actividad |

La regla que quedó fue: **lo que define el negocio y necesita reglas estrictas va en MySQL; lo que crece rápido o cambia de forma va en MongoDB.** El stock actual se dejó en MySQL a propósito, porque es el dato más crítico y debe haber una sola versión de él. MongoDB guarda la historia de cómo se llegó a ese número (algo así como el saldo de una cuenta y su extracto).

# 3. Estructura de las colecciones en MongoDB

Se usa una base documental llamada **inventario_gastrobar** con tres colecciones. No hay que crearlas a mano: MongoDB las crea con el primer documento. Los índices sí se crean desde el código al iniciar la app (archivo *extensions.py*).

## 3.1 fichas_producto

**Para qué:** guardar las características que dependen del tipo de producto. Hay un documento por producto.

```
{
  "_id": ObjectId("..."),
  "producto_id": 1,                 // id del producto en MySQL
  "tipo": "cerveza",
  "atributos": {
    "grados_alcohol": 4.7,
    "volumen_ml": 330,
    "estilo": "lager",
    "retornable": true
  },
  "etiquetas": ["nacional", "mas_pedida"],
  "notas": null,
  "actualizado": ISODate("2026-11-03T21:00:00Z")
}
```

Otro producto de la misma colección, con atributos totalmente distintos:

```
{ "producto_id": 8, "tipo": "comida",
  "atributos": { "requiere_refrigeracion": true, "vida_util_dias": 15,
                 "alergenos": ["soya"], "peso_g": 1000 } }
```

**Por qué NoSQL:** si esto se hiciera en SQL habría dos caminos malos. Uno es una tabla con columnas para todo (grados, ml, alérgenos, material, años de añejamiento...) donde casi todas quedan en NULL. El otro es un modelo entidad-atributo-valor, que obliga a guardar todo como texto y hacer muchos JOIN. Con un documento cada producto guarda solo lo que le aplica, y si mañana aparece un atributo nuevo (por ejemplo "sabor" para un ron) se agrega con un *$set* sin modificar ninguna estructura. El campo *atributos* se dejó como objeto anidado para separar lo variable de lo que sí es común (tipo, etiquetas). Las *etiquetas* son un arreglo, y Mongo permite buscar dentro de él directamente (*?etiqueta=importado*).

**Índice:** { producto_id: 1 } **único**, para que un producto no pueda tener dos fichas.

## 3.2 movimientos_inventario

**Para qué:** registrar cada cambio en el inventario. Es la colección que más crece: con solo nueve productos de prueba, 30 días de operación generaron más de 270 movimientos.

```
{
  "_id": ObjectId("..."),
  "producto_id": 1,
  "producto_nombre": "Cerveza Club Colombia Dorada 330ml",
  "tipo": "salida",                // entrada | salida | merma | ajuste
  "cantidad": 3,
  "stock_anterior": 123,
  "stock_resultante": 120,
  "usuario": "mesero1",
  "motivo": null,
  "detalle": { "mesa": 9, "comanda": 2662 },
  "fecha": ISODate("2026-11-03T02:08:36Z")
}
```

El campo *detalle* es el que cambia según el tipo de movimiento:

| Tipo | Ejemplo de detalle |
|---|---|
| salida (venta) | { "mesa": 4, "comanda": 5120 } o { "preparacion": "cocina" } |
| entrada (compra) | { "factura_proveedor": "FV-2231", "lote": "L5521" } |
| merma (pérdida) | { "causa": "botella rota" } |

**Por qué NoSQL:** (1) es información de alta frecuencia: se escribe mucho y casi nunca se modifica; (2) el detalle tiene forma distinta en cada tipo; (3) la consulta típica es "los últimos N movimientos de un producto" o "cuánto se vendió en los últimos días", que en Mongo se resuelven con un *find* ordenado o con un pipeline de agregación.

**Decisiones de diseño:**

- **producto_nombre duplicado (desnormalización):** así el historial se puede mostrar sin consultar MySQL, y si el producto se elimina el historial sigue siendo entendible.
- **stock_anterior y stock_resultante:** dejan ver cómo quedó el inventario después de cada movimiento sin recalcular nada; sirve para auditoría.
- **Se usa referencia (producto_id) y no se embeben los movimientos dentro del producto:** un producto puede tener miles de movimientos y un documento de Mongo tiene un límite de 16 MB; además sería un documento que se reescribe en cada venta.
- **La cantidad no se puede editar.** Si hay un error, el movimiento se anula (lo que revierte el stock en MySQL) y se registra uno nuevo. Solo se permite corregir *motivo* y *detalle*.

**Índice:** { producto_id: 1, fecha: -1 } compuesto, que es exactamente el orden en que se consultan los últimos movimientos de un producto.

### 3.3 logs_actividad

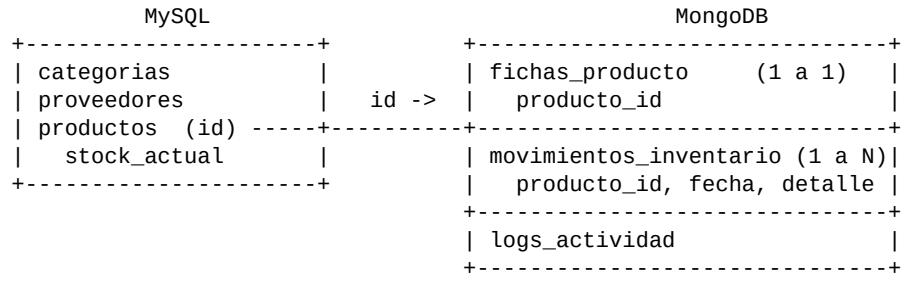
**Para qué:** dejar rastro de cada acción hecha en la API (crear, actualizar, eliminar) sobre cualquier entidad.

```
{ "accion": "ACTUALIZAR", "entidad": "producto", "entidad_id": 10,
  "usuario": "sistema", "datos": { "precio_venta": 98000 },
  "fecha": ISODate("2026-11-03T21:09:08Z") }
```

**Por qué NoSQL:** el campo *datos* guarda lo que cambió en cada acción y nunca tiene la misma forma. Además son registros que solo se insertan y se leen en orden de fecha. **Índice:** { fecha: -1 }.

## 4. Cómo se relacionan MySQL y MongoDB

MongoDB no tiene llaves foráneas, así que la relación la controla la aplicación. El puente es el campo **producto_id**, que corresponde al *id* de la tabla productos en MySQL:




- Antes de crear una ficha o un movimiento, la API verifica en MySQL que el producto exista (y para los movimientos, que esté activo).
- Al eliminar un producto en MySQL también se elimina su ficha en MongoDB. Los movimientos se conservan como historial.

## 5. Lógica de negocio que combina las dos bases

| Endpoint | MySQL | MongoDB |
|---|---|---|
| GET /api/productos/<id>/detalle | Producto, categoría, proveedor, alerta de stock | Ficha + últimos 10 movimientos |
| POST /api/movimientos | Valida producto y stock; actualiza stock_actual | Inserta el movimiento y el log |
| DELETE /api/movimientos/<id> | Revierte el stock | Elimina el movimiento |
| GET /api/reportes/stock-bajo | Productos con stock <= mínimo | Consumo de los últimos N días ($match + $group) |
| GET /api/reportes/mas-vendidos | Nombre, categoría, precio de venta | Ranking de salidas ($group + $sort + $limit) |

### 5.1 Consistencia al registrar un movimiento

El caso más delicado es registrar una venta, porque se escribe en las dos bases y no puede quedar una actualizada y la otra no. El orden que se usó es:

1. Se cambia el stock en la sesión de SQLAlchemy, **sin hacer commit**.
2. Se inserta el documento en MongoDB.
3. Si Mongo respondió bien se hace *commit* en MySQL; si falló se hace *rollback* y el stock queda como estaba.

Esto se probó simulando una caída de Mongo: la API respondió error 500 y el stock en MySQL siguió en el mismo valor. No es una transacción distribuida completa (si MySQL fallara justo después de guardar en Mongo quedaría un movimiento sin aplicar), pero para el tamaño de este negocio es suficiente y la caída de MySQL en ese punto es poco probable.

### 5.2 Ejemplo: reporte de stock bajo

Este reporte responde una pregunta real del negocio: *¿qué tengo que pedir y para cuántos días me alcanza?* MySQL dice qué productos están bajo el mínimo; MongoDB suma lo vendido en los últimos días con este pipeline:

```
[
  { "$match": { "producto_id": { "$in": [ids en alerta] },
                "tipo": "salida", "fecha": { "$gte": hace_7_dias } } },
  { "$group": { "_id": "$producto_id", "consumido": { "$sum": "$cantidad" } } }
]
```

Con eso la API calcula el consumo promedio diario y los días estimados que dura el stock actual, y ordena los productos del más urgente al menos urgente.

## 6. Conclusión

Separar los datos así deja a cada base haciendo lo que mejor hace. MySQL se queda con el catálogo y el stock, donde importan las relaciones y las reglas. MongoDB guarda lo que tiene forma variable o crece rápido: atributos de producto, historial de movimientos y logs. El costo es que la aplicación tiene que encargarse de mantener la relación entre las dos (validar el producto_id y coordinar commit y rollback), y eso quedó resuelto en la capa de rutas. A futuro, la colección de movimientos podría servir para predecir pedidos a proveedores o para detectar mermas fuera de lo normal.

**Repositorio:** https://github.com/MrHuesitozzz/inventario-gastrobar